

MYMANAGER

Toni Heikkilä
TAMPEREEN UNIVERSITY

Table of Contents

General Description	2
Installation.....	2
Usage	2
Structure of the Program.....	3
Secure Programming Solutions	3
OWASP TOP 10.....	3
Testing	4
Future implementations	5
Security issues and vulnerabilities	5
Use of AI	6

General Description

This program works as an offline CLI-based (Command Line Interface) password manager.

Installation

Program was tested mainly on Windows using Command Prompt. Instructions are based on that as other platforms were not tested. Most likely on other platforms .exe does not work and user needs to recompile the program for their interface. My compile command as reference:

```
g++ -std=c++17 ^  
main.cpp ^  
crypto/crypto_manager.cpp ^  
storage/file_storage.cpp ^  
app/vault_manager.cpp ^  
-I. ^  
-LC:/msys64/mingw64/lib ^  
-lsodium ^  
-o MyManager.exe
```

1. Start Command Prompt and create + navigate to the folder you wish to clone program into.
2. git clone <https://github.com/tonihoooo/COMP.SEC.300.git>
3. cd COMP.SEC.300
4. Start program with >MyManager.exe

Usage

Usage should be quite straightforward. Program asks for master password when started and on first use you set your master password. At the current state of the program this password must be remembered as password vault is crypted based on this and there is no way to decrypt the vault if this password is lost or forgotten. vault.dat is created on program folder which should be deleted if the master password is lost or there is another reason to reset the vault. Once master password is entered user has four commands to choose from: add, list, delete and exit.

add – Add a new password to the vault

1. Type “add”
2. Type <entry name>
3. Type <username>
4. type <password>
5. Entry saved.

list – List all the passwords currently stored in the vault

delete – Delete a password from the vault

1. Type “delete”
2. type <entry name>
3. Entry deleted

exit – Exit the vault and terminate the program

Program is restarted with starting the .exe and typing the same master password.

Structure of the Program

Main program logic is handled in app/, crypto/ and storage/. Program uses catch2 for testing and Libsodium for encryption/decryption.

main.cpp interacts with the following folders: app/, crypto/ and storage/. App encloses vault_manager.cpp which is responsible for main application logic. Crypto encloses cryptographic operations. Storage encloses file interaction.

CMakeLists.txt tells GitHub Actions how to build the program. github/workflows encloses ci.yml which has instructions for CI/CD pipeline. sonar-project.properties has instructions for SonarCloud.

./tests has all the tests for the program.

Secure Programming Solutions

The cryptographic choice of the program was Libsodium instead of manual custom crypto. Libsodium was found to be modern, well-tested and secure external library. It is used on encrypting the passwords before the data is written to the disk.

Program is clearly divided into different responsibility areas; app/, crypto/ and storage/. This way security-critical code is separated properly and maintainability is easier.

Static application security testing SAST is implemented through GitHub Actions CI/CD pipelines and SonarCloud. CI executes it automatically on every update.

Automated testing is done using Catch2. CI executes tests automatically through GitHub Actions.

In the code particular attention was paid not to hardcode any secrets to the code. Sensitive data is encrypted/decrypted dynamically during runtime. Password storage is hashed using nonce and random salt. Plaintext exposure in memory was minimized and program includes exception handling. Proper memory handling was kept in mind during programming.

OWASP TOP 10

A01: Broken Access Control

As program is local single-user CLI app this part is not applicable. Master password exists to protect vault access. Mostly not applicable.

A02: Security Misconfiguration

CI/CD and SonarCloud used.

A03: Software Supply Chain Failures

Dependencies installed through vckpg in GitHub Actions. CI builds from a clean environment. SonarCloud used for static analysis. Build artifacts/dependencies mostly excludes using .gitignore.

A04: Cryptographic Failures

Manual crypto replaced with Libsodium. Random salt, random nonce and authenticated encryption, decryption included.

A05: Injection

No SQL, no web input, no database. Mostly not applicable.

A06: Insecure Design

Layered design using app/, crypto/ and storage/.

A07: Authentication Failures

Master password is used and using wrong one fails safely. However, if master password is lost all data is lost.

A08: Software or Data Integrity Failures

Vault format has magic/version header. GitHub Actions builds and tests project. SonarCloud and SAST used. However, all data is lost if vault file gets corrupted.

A09: Security Logging and Alerting Failures

CLI has simple messages only. No proper logging exists atm which on the other hand avoids exposing secrets.

A10: Mishandling of Exceptional Conditions

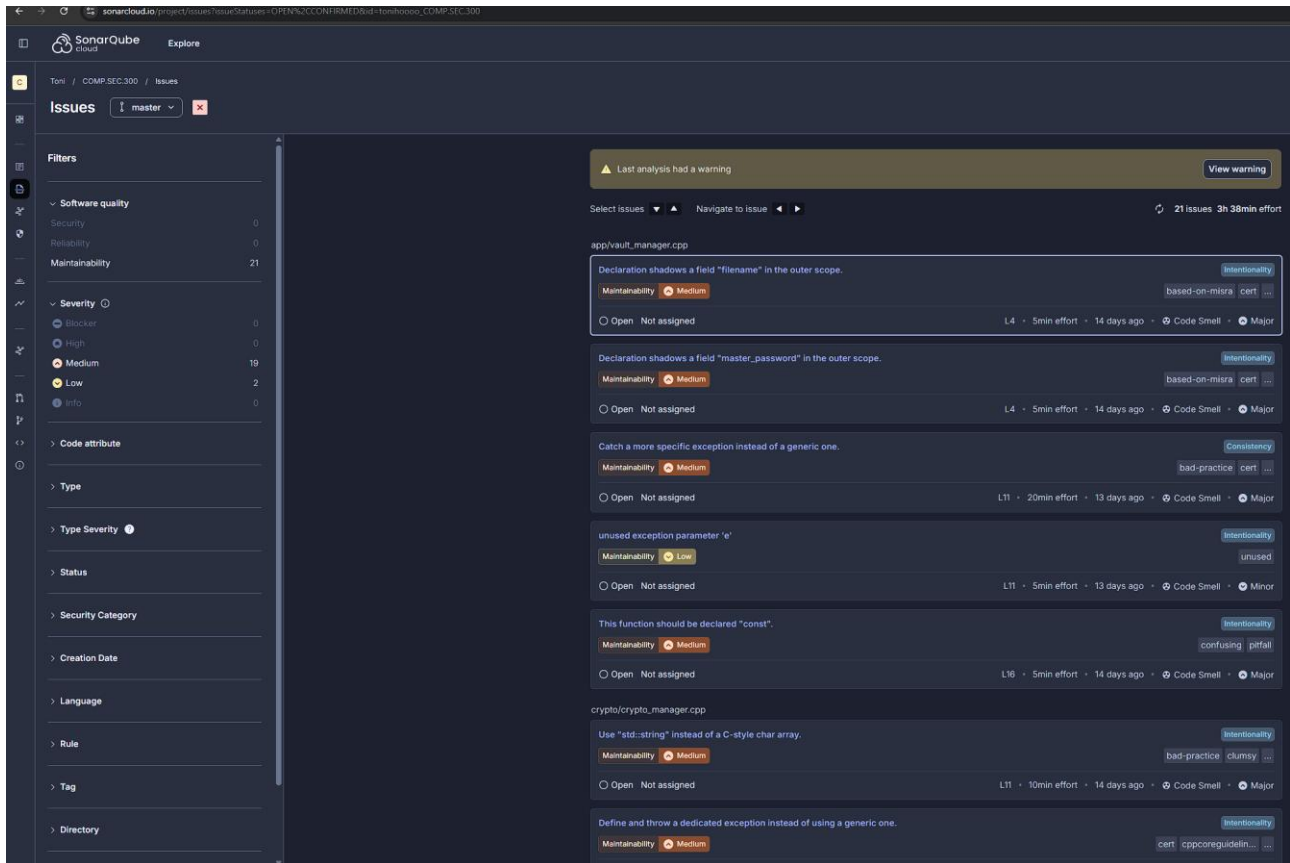
Wrong master password handling. Libsodium authentication failure handling. Loading corrupted vault handled. CLI shows some messages when exceptions arise.

Testing

Testing was done both manually and automatically. For automated tests Catch2 was used. Latest release of Catch2 was not used due to integration difficulties. The version of Catch2 currently in use is header-only which was relatively fast to implement. This is of course security issue which should be fixed but as there was not enough time for me to properly configure the latest version. SonarCloud is used for SAST.

Automated tests ran with Catch2 we're all successful. Latest successful run with SonarCloud included 19 medium issues and 2 low issues in the code. As there was no time to fix these these remained in the code. Below is a link to the results and a screenshot if link needs authentication.

https://sonarcloud.io/summary/new_code?id=tonihoooo_COMP.SEC.300&branch=master



Future implementations

Some future implementations for code would include adding MFA and prompting user to use secure master password. Current implementation has no requirements for master password. This prompting should be expanded for regular passwords as well. User input would require some fine-tuning as currently using spaces with commands like “add test test test” successfully adds new password as single-line input but it makes the UI look weird. This is not user-friendly and is quite fast to fix but as UX was not on primary focus on this course I decided to use my time to battle security and testing issues instead.

Project would require more thorough exception handling and more automated tests. Catch2 should be updated to the latest version. SonarCloud implementation does not function as it should. On the latest working version files in app/, crypto/ and storage/ we’re tested but main.cpp and tests/ we’re not as including tests/ caused errors which I could not fix in the given time window. CI/CD pipeline only has partly functioning SAST test now. I wanted to add at least SCA and automatic DAST but I ran out of time.

Security issues and vulnerabilities

Big security issue itself is the fact that project is missing SCA and DAST completely. Some minor DAST testing was done manually, but it should be automated. Also not using the latest

version of Catch2 is a major security issue. However, as project only runs locally and offline it requires an attack originating from user's own hardware. These should be fixed, nevertheless.

SonarCloud shows 19 medium issues and 2 low issues.

Master password or passwords in general are not prompted to be secure enough now; this should be addressed to the user.

Use of AI

AI was used as a development tool for programming in this course. AI in use was mainly ChatGPT's latest version GPT-5.5, but the latest version of GitHub Copilot on VSCode was also tested. However, as GitHub Copilot was quite slow and unhelpful I think none of the information it gave is used on this project.

GPT-5.5 was used for troubleshooting any issues on the code, for example the error output of Command Prompt when program was executed locally. Exception information was given to the AI and on the later stages the build errors arisen in GitHub actions. AI was used together with external library documentation to produce some of the code, for example for the parts with Libsodium where encryption and decryption was done. The reason for this was that it was much faster than using the documentation only as I had not used the library before. All the code suggestions and other proposals from AI we're manually reviewed especially taking security into account before being implemented.